PROYECTO FINAL · DESARROLLO WEB FULL-STACK

Sabías Que PR

Planificación y Diseño de la Base de Datos

Fase 1: Planificación y Diseño · Fase 2: Diseño de la Base de Datos

Estudiante	William Collado
Curso	COMP2053 — Desarrollo Web Full-Stack
Institución	Universidad Interamericana de Puerto Rico, Recinto de Fajardo
Repositorio	github.com/Wcollado72/SabiasquePR
Sitio en vivo	fabulous-tartufo-fcb839.netlify.app
Fecha	17 de septiembre de 2026 (actualizado)

FASE 1 · PLANIFICACIÓN Y DISEÑO

1. Descripción del proyecto

Sabías Que PR es una plataforma web informativa y comunitaria, nacida en Puerto Rico con una misión sin fronteras: cualquier persona, en cualquier país, que quiera entender su entorno y ser agente de cambio. Combina noticias categorizadas con contexto histórico y análisis crítico, contenido educativo organizado por rutas de aprendizaje, una tienda de artículos que apoya el sostenimiento del proyecto, y un foro comunitario donde los usuarios registrados publican y comentan — con un modelo de suscripción premium que amplía lo que pueden publicar.

El sitio nació como una aplicación estática (13 páginas HTML generadas por un script de plantillas propio) y, como proyecto final del curso, evolucionó hacia una aplicación **full-stack** real: un frontend en HTML5/CSS3/JavaScript que consume una **API REST propia** construida en Node.js y Express, con persistencia en una base de datos **PostgreSQL** (alojada en Supabase), autenticación de usuarios mediante **JSON Web Tokens verificados en el servidor** — no solo en el navegador — y suscripciones recurrentes procesadas por **Stripe**. La aplicación ya está **desplegada y accesible públicamente** (Netlify + Render), no solo en un entorno local.

2. Objetivo general

Diseñar, desarrollar, probar y desplegar una aplicación web full-stack funcional para el proyecto **Sabías Que PR**, que integre un frontend responsivo, una API REST propia con operaciones CRUD completas, una base de datos relacional persistente y un sistema de autenticación con JWT verificado en el servidor, aplicando de manera integrada las tecnologías y metodologías estudiadas durante el curso de Desarrollo Web Full-Stack.

3. Justificación

Antes de este proyecto, todo lo "dinámico" del sitio dependía de que el navegador llamara directamente a servicios de terceros sin configurar (Formspree, Supabase, Mailchimp, Stripe), y el contenido se editaba a mano en archivos JSON. Un proyecto final de desarrollo full-stack evalúa que el equipo construya y controle las tres capas de una aplicación real: un backend/API propio, una base de datos con esquema propio, y un frontend que consuma esa API — no solo enlaces a widgets externos.

Construir un **foro comunitario real** como eje del CRUD, junto con la gestión de noticias y productos vía API, resuelve ambos objetivos a la vez: cumple el criterio de mayor peso de la rúbrica del curso (30%, funcionalidad y CRUD completo) y avanza la visión original del proyecto de convertirse en un espacio donde la comunidad no solo lee, sino que participa.

4. Alcance del sistema

Incluido en esta entrega:

- Foro comunitario con CRUD completo (publicaciones y comentarios), filtrable por categoría, con límite de palabras según el rol del usuario (100 para usuarios regulares, 500 para premium), autenticación JWT verificada en el servidor y permisos de dueño/administrador.
- CRUD vía API REST propia para noticias (filtrables por 11 categorías) y productos (antes generados estáticamente por un script de plantillas).
- Autenticación completa de extremo a extremo: registro, inicio de sesión, recuperación de contraseña e inicio de sesión con Google (Supabase Auth), verificada también del lado del servidor, con acceso disponible desde una ventana de inicio de sesión en todas las páginas.
- Suscripciones premium recurrentes vía Stripe (checkout + webhook que sincroniza el rol del usuario automáticamente) — construidas y funcionando en modo de prueba (sandbox), sin cobrar dinero real todavía.
- Panel de administración protegido por rol, con formularios CRUD sobre noticias (incluyendo categoría) y productos.
- Suite de pruebas automatizadas (94 pruebas, unitarias e integración) sobre la API.
- Despliegue público real de la aplicación (frontend en Netlify, backend en Render) en una URL accesible para evaluación, con un flujo automatizado (GitHub Actions) que mantiene la base de datos activa.

Fuera de alcance para esta entrega académica (parte del roadmap de producto, no del criterio de evaluación):

- Activar Stripe en modo real (llaves de producción) — las suscripciones premium están construidas y probadas, pero solo con llaves de prueba.
- Checkout de carrito multi-producto para la tienda (hoy usa enlaces de pago individuales de Stripe por producto).
- Sistema de newsletter propio para reemplazar Mailchimp.
- Conexión al dominio de producción real (sabiasquepr.com) — deliberadamente pospuesta hasta tener más contenido publicado; el despliegue académico usa la URL de Netlify/Render, que ya es pública y funcional.
- Catalogador de noticias asistido por IA, sección de videos y soporte multi-idioma — roadmap de producto a futuro.

5. Tecnologías seleccionadas

Capa	Tecnología	Uso en el proyecto
Frontend	HTML5, CSS3, JavaScript (ES modules)	Sitio responsive, modo oscuro con persistencia, PWA (manifest + service worker)
Backend	Node.js + Express	API REST propia (backend/src) con middleware de validación y manejo de errores centralizado
Base de datos	PostgreSQL (Supabase)	Esquema relacional propio con Row Level Security activo
Autenticación	Supabase Auth + jose (JWKS, ES256)	Login/registro/OAuth en el cliente; verificación de JWT en cada ruta protegida del servidor
Pagos	Stripe (suscripciones)	Checkout de suscripción premium + webhook que actualiza el rol del usuario automáticamente (modo prueba)

Pruebas	Jest + Supertest	94 pruebas unitarias de validadores/middleware y de integración de la API
Control de versiones	Git + GitHub	Historial de commits por fase (github.com/Wcollado72/SabiasquePR)
Despliegue	Netlify + Render	Frontend estático (Netlify) y backend Express (Render) — ya desplegados y accesibles públicamente
Automatización	GitHub Actions	Ping periódico a la API (cada 3 días) para evitar que Supabase pause la base de datos por inactividad

6. Diagrama de arquitectura

El navegador carga el frontend estático y, por separado, se comunica directamente con Supabase Auth para autenticarse (obteniendo un JWT firmado). El frontend consulta el backend propio mediante fetch, enviando ese JWT en el encabezado `Authorization: Bearer`. El backend valida el JWT contra el JWKS público de Supabase (sin necesidad de compartir ningún secreto) y solo entonces consulta PostgreSQL.

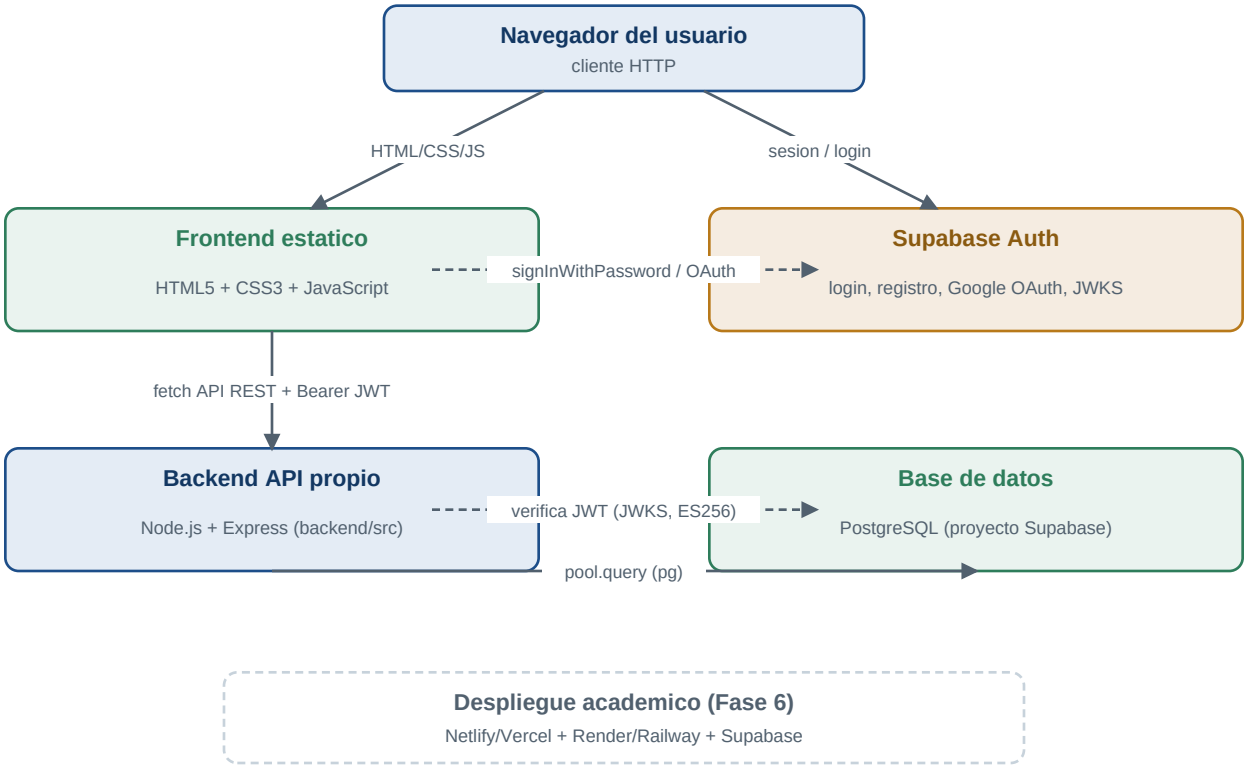


Figura 1. Arquitectura de tres capas: frontend, API propia y base de datos, con Supabase Auth como servicio de identidad compartido.

Un cuarto integrante, no mostrado en el diagrama por espacio, es **Stripe**: cuando un usuario se suscribe a premium desde `premium.html`, el backend crea una sesión de checkout de Stripe; al completarse el pago, Stripe llama a un *webhook* propio (`POST /api/stripe/webhook`), verificado con la firma de Stripe, que actualiza directamente el rol del usuario en PostgreSQL — sin que el frontend tenga que hacer nada adicional.

7. Diagrama general de navegación

Todas las páginas públicas comparten la barra de navegación superior y un botón de inicio de sesión en el encabezado (ventana emergente, disponible en toda página). El foro permite lectura pública y restringe la escritura (publicar, comentar) a usuarios con sesión activa; el panel de administración está completamente restringido al rol `admin`; la página Premium inicia el checkout de Stripe para quien quiera suscribirse. Las páginas "Sobre el proyecto", "Crítica Internacional" y "Tecnología" se retiraron: la primera se fusionó dentro de Inicio, y los temas de las otras dos se absorbieron en las categorías del feed de Noticias.

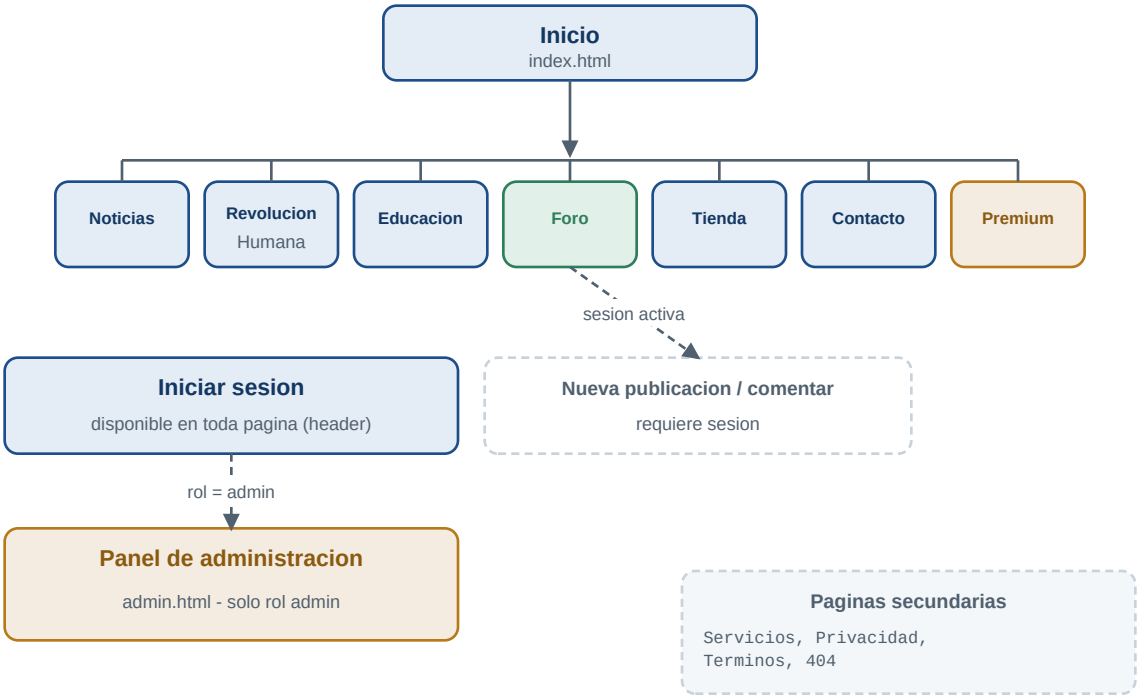
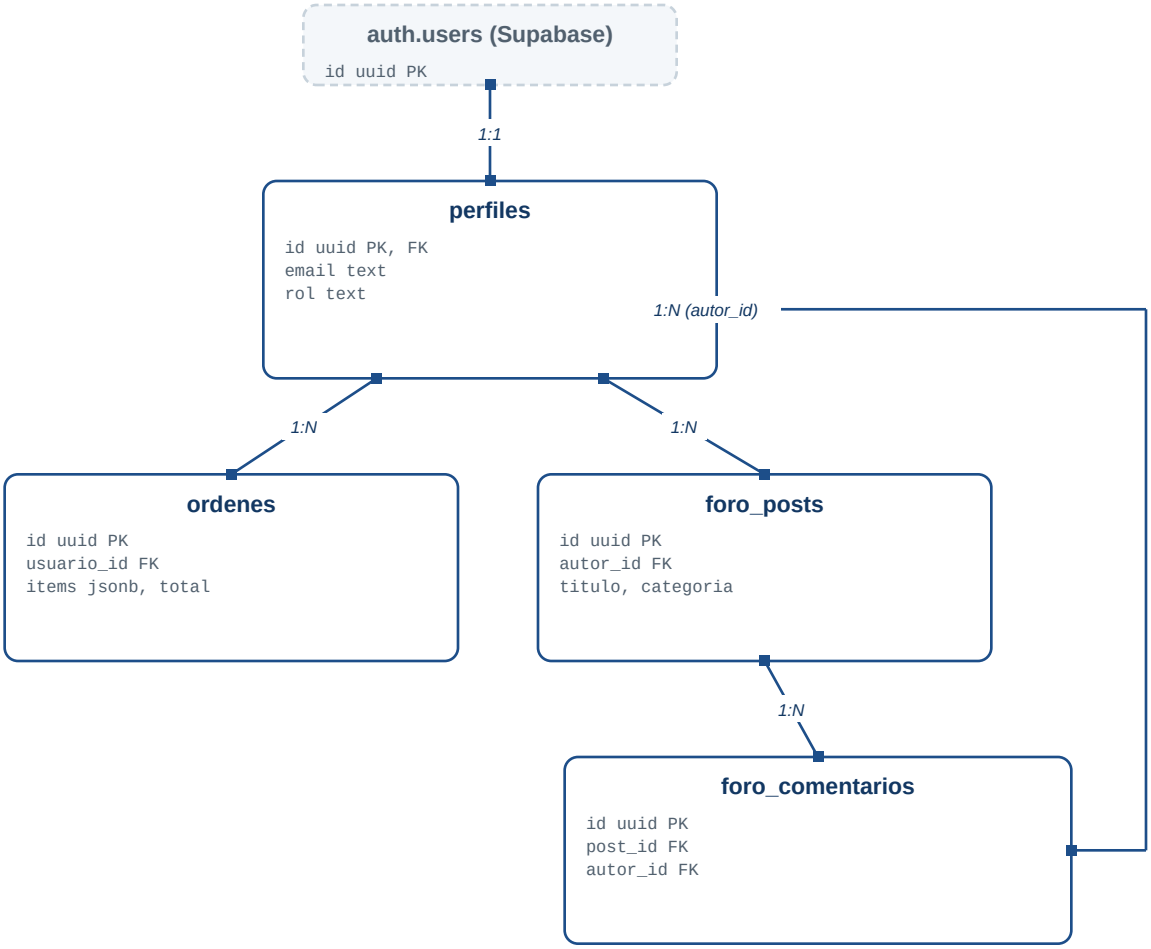


Figura 2. Mapa de navegación del sitio: páginas públicas, área que requiere sesión (foro) y área restringida por rol (panel de administración).

8. Modelo Entidad-Relación (MER)

La base de datos vive en el proyecto de Supabase (PostgreSQL administrado). `auth.users` es la tabla gestionada por Supabase Auth; `perfiles` extiende esa identidad con el rol de autorización (`usuario` / `admin`) y es el punto de origen de todas las relaciones de autoría del sistema. Todas las tablas tienen **Row Level Security** activo.



Entidades independientes (sin relaciones foraneas)

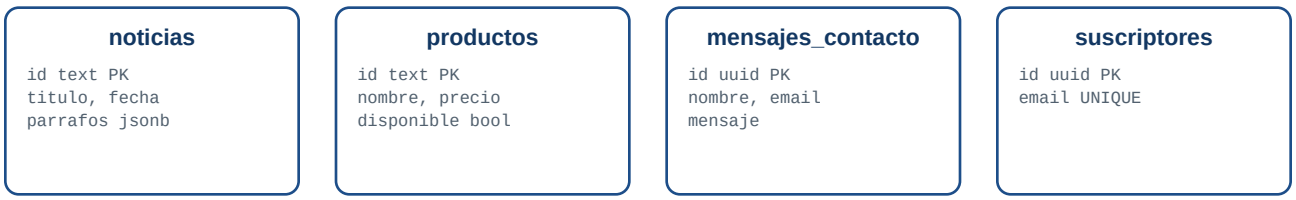


Figura 3. Modelo Entidad-Relación: 8 tablas propias más la tabla de identidad de Supabase Auth (externa, gestionada por la plataforma).

9. Esquema relacional y diccionario de datos

A continuación, el diccionario de datos completo: cada tabla, sus columnas, tipo de dato, restricciones (llaves primarias/foráneas, valores por defecto, validaciones) y una descripción de su propósito. Corresponde exactamente al script `backend/sql/schema.sql` del repositorio.

noticias

Campo	Tipo	Restricciones	Descripcion
id	text	PK	Slug unico generado del titulo
titulo	text	NOT NULL	Titular de la noticia
fecha	date	NOT NULL	Fecha editorial de la noticia
fecha_display	text	NOT NULL	Fecha en formato legible (es-PR)
imagen	text	NOT NULL	Ruta o URL de la imagen principal
imagen_alt	text	NOT NULL	Texto alternativo (accesibilidad)
imagen_caption	text	NOT NULL	Pie de foto / credito
clase	text	NOT NULL	Clase CSS para estilos particulares
fuente_url	text	NOT NULL	URL de la fuente original
fuente_texto	text	NOT NULL	Nombre visible de la fuente
parrafos	jsonb	NOT NULL, DEFAULT '[]'	Arreglo de parrafos del cuerpo
categoria	text	NOT NULL, DEFAULT 'general', CHECK IN (11 valores)	Categoria tematica (filtrable desde el frontend)
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de creacion del registro

productos

Campo	Tipo	Restricciones	Descripcion
id	text	PK	Slug unico generado del nombre
nombre	text	NOT NULL	Nombre del producto
descripcion	text	NOT NULL	Descripcion corta
precio	numeric(10, 2)	NOT NULL	Precio en USD
imagen	text	NOT NULL	Ruta o URL de la imagen
imagen_alt	text	NOT NULL	Texto alternativo
stripe_link	text	NULL	Enlace de pago de Stripe (Fase 5 del roadmap de producto)
disponible	boolean	NOT NULL, DEFAULT true	Visible en la tienda

creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de creacion del registro

perfiles

Campo	Tipo	Restricciones	Descripcion
id	uuid	PK, FK -> auth.users(id) ON DELETE CASCADE	Mismo id que el usuario de Supabase Auth
email	text	NOT NULL	Correo del usuario
rol	text	NOT NULL, DEFAULT 'usuario', CHECK IN ('usuario','admin','premium')	Rol de autorizacion (premium ampliado por Stripe)
stripe_customer_id	text	NULL, UNIQUE (parcial)	Id del cliente en Stripe, una vez se suscribe
stripe_subscription_id	text	NULL	Id de la suscripcion activa en Stripe
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de creacion del perfil

mensajes_contacto

Campo	Tipo	Restricciones	Descripcion
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador del mensaje
nombre	text	NOT NULL	Nombre de quien escribe
email	text	NOT NULL	Correo de contacto
motivo	text	NOT NULL	Motivo del mensaje
mensaje	text	NOT NULL	Contenido del mensaje
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de envio

suscriptores

Campo	Tipo	Restricciones	Descripcion
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador del suscriptor
email	text	NOT NULL, UNIQUE	Correo suscrito al boletin
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de suscripcion

ordenes

Campo	Tipo	Restricciones	Descripcion
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador de la orden
usuario_id	uuid	FK -> perfiles(id) ON DELETE SET NULL	Comprador (si inicio sesion)

items	jsonb	NOT NULL	Lista de productos y cantidades
total	numeric(10, 2)	NOT NULL	Total cobrado
stripe_session_id	text	NULL	Referencia de la sesion de pago
estado	text	NOT NULL, DEFAULT 'pendiente', CHECK IN ('pendiente','pagada','cancelada')	Estado del pago
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de la orden

foro_posts

Campo	Tipo	Restricciones	Descripcion
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador de la publicacion
autor_id	uuid	NOT NULL, FK -> perfiles(id) ON DELETE CASCADE	Autor de la publicacion
titulo	text	NOT NULL	Titulo de la publicacion
cuerpo	text	NOT NULL	Contenido de la publicacion
categoria	text	NOT NULL, DEFAULT 'general', CHECK IN (5 valores)	Categoria tematica
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de publicacion
editado_en	timestamptz	NULL	Fecha de la ultima edicion

foro_comentarios

Campo	Tipo	Restricciones	Descripcion
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador del comentario
post_id	uuid	NOT NULL, FK -> foro_posts(id) ON DELETE CASCADE	Publicacion comentada
autor_id	uuid	NOT NULL, FK -> perfiles(id) ON DELETE CASCADE	Autor del comentario
cuerpo	text	NOT NULL	Contenido del comentario
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha del comentario

10. Normalización e índices

El esquema está normalizado a Tercera Forma Normal (3FN): cada tabla describe una sola entidad, sin grupos repetitivos ni dependencias transitivas. La única excepción deliberada es `noticias.parrafos` y `ordenes.items`, almacenados como `jsonb` por tratarse de listas ordenadas sin necesidad de consultarse

individualmente por fuera de su registro padre.

- `idx_foro_posts_categoria` — acelera el filtrado del foro por categoría, ordenado por fecha.
- `idx_foro_comentarios_post_id` — acelera la carga de comentarios de una publicación.
- Restricciones `CHECK` en `perfiles.rol`, `foro_posts.categoria` y `ordenes.estado` sustituyen tablas de catálogo separadas, dado el número reducido y fijo de valores válidos.

11. Script de creación de la base de datos

El script completo de creación (DDL) vive en el repositorio en `backend/sql/schema.sql` y se aplica con `npm run db:schema` (backend/scripts/apply-schema.js). Fue verificado aplicándolo contra el proyecto real de Supabase durante la Fase 1 del desarrollo. Se incluye a continuación tal como está en el repositorio.

```
-- Esquema inicial de Sabias que PR
-- Fase 1: solo estructura. Los endpoints CRUD se conectan en la Fase 2.

CREATE TABLE IF NOT EXISTS noticias (
  id              text PRIMARY KEY,
  titulo          text NOT NULL,
  fecha           date NOT NULL,
  fecha_display   text NOT NULL,
  imagen          text NOT NULL,
  imagen_alt      text NOT NULL,
  imagen_caption  text NOT NULL,
  clase           text NOT NULL,
  fuente_url      text NOT NULL,
  fuente_texto    text NOT NULL,
  parrafos        jsonb NOT NULL DEFAULT '[]',
  creado_en       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS productos (
  id              text PRIMARY KEY,
  nombre          text NOT NULL,
  descripcion     text NOT NULL,
  precio          numeric(10,2) NOT NULL,
  imagen          text NOT NULL,
  imagen_alt      text NOT NULL,
  stripe_link     text,
  disponible      boolean NOT NULL DEFAULT true,
  creado_en       timestamptz NOT NULL DEFAULT now()
);

-- Perfil extendido de cada usuario de Supabase Auth (auth.users)
CREATE TABLE IF NOT EXISTS perfiles (
  id              uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email           text NOT NULL,
  rol             text NOT NULL DEFAULT 'usuario' CHECK (rol IN ('usuario', 'admin')),
  creado_en       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS mensajes_contacto (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  nombre          text NOT NULL,
  email           text NOT NULL,
  motivo          text NOT NULL,
  mensaje         text NOT NULL,
  creado_en       timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE IF NOT EXISTS suscriptores (
```

```
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email           text NOT NULL UNIQUE,
  creado_en       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS ordenes (
  id                  uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  usuario_id          uuid REFERENCES perfiles(id) ON DELETE SET NULL,
  items               jsonb NOT NULL,
  total               numeric(10,2) NOT NULL,
  stripe_session_id   text,
  estado              text NOT NULL DEFAULT 'pendiente' CHECK (estado IN ('pendiente', 'pagada', 'cancelada')),
  creado_en           timestamptz NOT NULL DEFAULT now()
);

-- Publicaciones del foro comunitario
CREATE TABLE IF NOT EXISTS foro_posts (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  autor_id        uuid NOT NULL REFERENCES perfiles(id) ON DELETE CASCADE,
  titulo          text NOT NULL,
  cuerpo          text NOT NULL,
  categoria       text NOT NULL DEFAULT 'general'
                    CHECK (categoria IN ('general','tecnologia','internacionales','nuevos-humanos','educacion')),
  creado_en       timestamptz NOT NULL DEFAULT now(),
  editado_en      timestamptz
);

-- Comentarios de cada publicacion
CREATE TABLE IF NOT EXISTS foro_comentarios (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  post_id         uuid NOT NULL REFERENCES foro_posts(id) ON DELETE CASCADE,
  autor_id        uuid NOT NULL REFERENCES perfiles(id) ON DELETE CASCADE,
  cuerpo          text NOT NULL,
  creado_en       timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_foro_posts_categoria      ON foro_posts(categoria, creado_en DESC);
CREATE INDEX IF NOT EXISTS idx_foro_comentarios_post_id  ON foro_comentarios(post_id, creado_en ASC);

-- Suscripciones premium (Stripe). Se agregan via ALTER porque
-- CREATE TABLE IF NOT EXISTS no modifica una tabla ya existente.
ALTER TABLE perfiles ADD COLUMN IF NOT EXISTS stripe_customer_id     text;
ALTER TABLE perfiles ADD COLUMN IF NOT EXISTS stripe_subscription_id text;

-- Ampliar el rol permitido para incluir 'premium'.
ALTER TABLE perfiles DROP CONSTRAINT IF EXISTS perfiles_rol_check;
ALTER TABLE perfiles ADD CONSTRAINT perfiles_rol_check
  CHECK (rol IN ('usuario', 'admin', 'premium'));

CREATE UNIQUE INDEX IF NOT EXISTS idx_perfiles_stripe_customer_id
  ON perfiles(stripe_customer_id) WHERE stripe_customer_id IS NOT NULL;

-- Categoria de cada noticia (separada de 'clase', que es solo para
-- estilos CSS). Permite filtrar el feed de Noticias por tema.
ALTER TABLE noticias ADD COLUMN IF NOT EXISTS categoria text NOT NULL DEFAULT 'general';

ALTER TABLE noticias DROP CONSTRAINT IF EXISTS noticias_categoria_check;
ALTER TABLE noticias ADD CONSTRAINT noticias_categoria_check
  CHECK (categoria IN (
    'general', 'internacionales', 'crimenes', 'salud', 'deportes',
    'finanzas', 'tecnologia', 'descubrimientos', 'politica', 'clima', 'cultura'
  ));

CREATE INDEX IF NOT EXISTS idx_noticias_categoria ON noticias(categoria, fecha DESC);

-- La pagina "Critica Internacional" se retiro; sus temas se fusionaron
-- en la categoria "internacionales". Reasignar publicaciones existentes
-- antes de endurecer el CHECK (el CREATE TABLE de arriba no modifica una
```

```
-- tabla ya existente).
UPDATE foro_posts SET categoria = 'internacionales' WHERE categoria = 'critica-internacional';

ALTER TABLE foro_posts DROP CONSTRAINT IF EXISTS foro_posts_categoria_check;
ALTER TABLE foro_posts ADD CONSTRAINT foro_posts_categoria_check
  CHECK (categoria IN ('general', 'tecnologia', 'internacionales', 'nuevos-humanos', 'educacion'));

-- RLS activo por defecto en todas las tablas (sin policies = sin acceso
-- publico via la Data API de Supabase; el backend usa la conexion directa
-- de Postgres y no pasa por RLS).
ALTER TABLE noticias           ENABLE ROW LEVEL SECURITY;
ALTER TABLE productos          ENABLE ROW LEVEL SECURITY;
ALTER TABLE perfiles           ENABLE ROW LEVEL SECURITY;
ALTER TABLE mensajes_contacto  ENABLE ROW LEVEL SECURITY;
ALTER TABLE suscriptores       ENABLE ROW LEVEL SECURITY;
ALTER TABLE ordenes            ENABLE ROW LEVEL SECURITY;
ALTER TABLE foro_posts         ENABLE ROW LEVEL SECURITY;
ALTER TABLE foro_comentarios   ENABLE ROW LEVEL SECURITY;
```